CHAPTER 11

MATRIX ALGORITHMS AND GRAPH PARTITIONING

A discussion of network algorithms that use matrix and linear algebra methods, including algorithms for partitioning network nodes into groups

IN THE preceding chapter we discussed a variety of computer algorithms for calculating quantities of interest on networks, including degrees, centralities, shortest paths, and connectivity. We continue our study of network algorithms in this chapter with algorithms based on matrix calculations and methods of linear algebra applied to the adjacency matrix or other network matrices such as the graph Laplacian. We begin with a simple example, the calculation of eigenvector centrality, which involves finding the leading eigenvector of the adjacency matrix, and then we move on to some more advanced examples, including Fiedler's spectral partitioning method and algorithms for network community detection.

11.1 LEADING EIGENVECTORS AND EIGENVECTOR CENTRALITY

As discussed in Section 7.2, the eigenvector centrality of a vertex i in a network is defined to be the ith element of the leading eigenvector of the adjacency matrix, meaning the eigenvector corresponding to the largest (most positive) eigenvalue. Eigenvector centrality is an example of a quantity that can be calculated by a computer in a number of different ways, but not all of them are equally efficient. One way to calculate it would be to use a standard linear algebra method to calculate the complete set of eigenvectors of the adjacency matrix, and then discard all of them except the one corresponding to the largest eigenvalue. This, however, would be a wasteful approach, since it involves calculating a lot of things that we don't need. A simpler and faster method for calculating the eigenvector centrality is the *power method*.

If we start with essentially any initial vector $\mathbf{x}(0)$ and multiply it repeatedly by the adjacency matrix $\mathbf{A}$, we get

$$\mathbf{x}(t) = \mathbf{A}^t \mathbf{x}(0), \tag{11.1}$$

and, as shown in Section 7.2, $\mathbf{x}(t)$ will converge[162] to the required leading eigenvector of $\mathbf{A}$ as $t \to \infty$. This is the power method, and, simple though it is, there is no faster method known for calculating the eigenvector centrality (or the leading eigenvector of any matrix). There are a few caveats, however:

1. The method will not work if the initial vector $\mathbf{x}(0)$ happens to be orthogonal to the leading eigenvector. One simple way to avoid this problem is to choose the initial vector to have all elements positive. This works because all elements of the leading eigenvector of a real matrix with non-negative elements have the same sign,[163] which means that any vector orthogonal to the leading eigenvector must contain both positive and negative elements. Hence, if we choose all elements of our initial vector to be positive, we are guaranteed that the vector cannot be orthogonal to the leading eigenvector.
2. The elements of the vector have a tendency to grow on each iteration—they get multiplied by approximately a factor of the leading eigenvalue each time, which is usually greater than 1. Computers however cannot handle arbitrarily large numbers. Eventually the variables storing the elements of the vector will overflow their allowed range. To obviate this problem, we must periodically renormalize the vector by dividing all the elements by the same value, which we are allowed to do since an eigenvector divided throughout by a constant is still an eigenvector. Any suitable divisor will do, but we might, for instance, divide by the magnitude of the vector, thereby normalizing it so that its new magnitude is 1.
3. How long do we need to go on multiplying by the adjacency matrix before the result converges to the leading eigenvalue? This will depend on how accurate an answer we require, but one simple way to gauge convergence is to perform the calculation in parallel for two different initial vectors and watch to see when they reach the same value, within some prescribed tolerance. This scheme works best if, for the particular initial vectors chosen, at least some elements of the vector converge to the final answer from opposite

directions for the two vectors, one from above and one from below. (We must make the comparisons immediately after the renormalization of the vector described in (2) above—if we compare unnormalized vectors, then most likely all elements will increase on every iteration and no convergence will be visible.) If we can find some elements that do this (and we usually can), then it is a fairly safe bet that the difference between the two values for such an element is greater than the difference of either from the true value of the same element in the leading eigenvector.

The power method can also be used to calculate the leading eigen*value* κ_1 of the adjacency matrix. Once the algorithm has converged to the leading eigenvector, one more multiplication by the adjacency matrix will multiply that vector by exactly a factor of κ_1. Thus, we can take the ratio of the values of any element of the vector at two successive iterations of the algorithm after convergence and that ratio should equal κ_1. Or we could take the average of the ratios for several different elements to reduce numerical errors. (We should however avoid elements whose values are very small, since a small error in such an element could lead to a large fractional error in the ratio; our accuracy will be better if we take the average of some of the larger elements.)

11.1.1 COMPUTATIONAL COMPLEXITY

How long does the power method take to run? The answer comes in two parts. First, we need to know how long each multiplication by the adjacency matrix takes, and second we need to know how many multiplications are needed to get a required degree of accuracy in our answer.

If our network is stored in adjacency matrix form, then multiplying that matrix into a given vector is straightforward. Exactly n^2 multiplications are needed for one matrix multiplication—one for each element of the adjacency matrix. We can do better, however, if our network is in adjacency list form. Elements of the adjacency matrix that are zero contribute nothing to the matrix multiplication and so can be neglected. The adjacency list allows us to skip the zero terms automatically, since it stores only the non-zero ones anyway.

In an ordinary unweighted network each non-zero element of the adjacency matrix is equal to 1. Let $\{u_j\}$, $j = 1 \ldots k_i$ be the set of neighbors of vertex i (where k_i is the degree of i). Then the ith element of $\mathbf{Ax}$, which we denote $[\mathbf{Ax}]_i$, is given by $[\mathbf{Ax}]_i = \sum_{j=1}^{k_i} x_{u_j}$. The evaluation of this sum involves only k_i operations, so one element of the matrix multiplication can be completed in time proportional to k_i and all elements can be completed in time proportional to $\sum_i k_i = 2m$, where m is the total number of edges in the network, or in other words in $O(m)$ time.

And how many such multiplications must we perform? Equation (7.4) tells us that after t iterations our vector is equal to

$$\mathbf{x}(t) = \kappa_1^t \sum_{i=1}^{n} c_i \left[\frac{\kappa_i}{\kappa_1}\right]^t \mathbf{v}_i,$$

(11.2)

where $\mathbf{v}_i$ is the normalized ith eigenvector, κ_i is the corresponding eigenvalue, and the c_i are constants whose values depend on the choice of initial vector. Rearranging slightly, we can write this as

$$\frac{\mathbf{x}(t)}{c_1\kappa_1^t} = \mathbf{v}_1 + \frac{c_2}{c_1}\left(\frac{\kappa_2}{\kappa_1}\right)^t \mathbf{v}_2 + \ldots,$$

(11.3)

which gives us our estimate of the leading eigenvector $\mathbf{v}_1$ plus the dominant contribution to the error. Neglecting the smaller terms, the root-mean-square error on the eigenvector is then

$$\sqrt{\left|\frac{\mathbf{x}(t)}{c_1\kappa_1^t} - \mathbf{v}_1\right|^2} = \frac{c_2}{c_1}\left(\frac{\kappa_2}{\kappa_1}\right)^t,$$

(11.4)

and if we want this error to be at most $\in$ then we require

$$t \geq \frac{\ln(1/\epsilon) + \ln(c_1/c_2)}{\ln(\kappa_1/\kappa_2)}.$$

(11.5)

Neither $\in$ nor the constants c_1 and c_2 depend on the network size. All the variation in the run time comes from the eigenvalues κ_1 and κ_2. The eigenvalues range in value from a maximum of κ_1 to a minimum of $\kappa_n \geq -|\kappa_1|$ and hence have a mean spacing of at most $2\kappa_1/(n - 1)$. Thus an order-of-magnitude estimate for the second eigenvalue is $\kappa_2 \simeq \kappa_1 - a\kappa_1/n$, where a is a constant of order unity, and hence

$$\ln \frac{\kappa_1}{\kappa_2} \simeq -\ln\left(1 - \frac{a}{n}\right) = \frac{a}{n} + O(n^{-2}).$$

(11.6)

Combining Eqs. (11.5) and (11.6), we find that the number of steps required for convergence of the power method is $t = O(n)$ to leading order.[164]

Overall therefore, the complete calculation of the eigenvector centralities of all n vertices of the network takes $O(n)$ multiplications which take $O(m)$ time each, or $O(mn)$ time overall, for a network stored in adjacency list format. If our network is sparse with $m \propto n$, a running time of $O(mn)$ is equivalent to $O(n^2)$. On the other hand, if the network is dense, with $m \propto n^2$, then $O(mn)$ is equivalent to $O(n^{164})$.

Conversely, if our network is stored in adjacency matrix format the multiplications take $O(n^2)$ time, as noted above, so the complete calculation takes $O(n^{164})$, regardless of whether the network is sparse or dense. Thus for the common case of a sparse matrix the adjacency list is the representation of choice for this calculation.

11.1.2 CALCULATING OTHER EIGENVALUES AND EIGENVECTORS

The power method of the previous section calculates the largest eigenvalue of a matrix and the corresponding eigenvector. This is probably the most common type of eigenvector calculation encountered in the study of networks, but there are cases where we wish to know other eigenvectors or eigenvalues as well. One example is the calculation of the so-called algebraic connectivity, which is the second smallest (or second most negative) eigenvalue of the graph Laplacian. As we saw in Section 6.13.3, the algebraic connectivity is non-zero if and only if a network is connected (i.e., has just a single component). The algebraic connectivity also appears in Section 11.5 as a measure of how easily a network can be bisected into two sets of vertices such that only a small number of edges run between the sets. Moreover, as we will see the elements of the corresponding eigenvector of the Laplacian tell us exactly how that bisection should be performed. Thus it will be useful to us to have a method for calculating eigenvalues beyond the largest one and their accompanying eigenvectors.

There are a number of techniques that can be used to find non-leading eigenvalues and eigenvectors of matrices. For instance, we can calculate the eigenvector corresponding to the most negative eigenvalue by shifting all the eigenvalues by a constant amount so that the most negative one becomes the eigenvalue of largest magnitude. The eigenvalues of the graph Laplacian $\mathbf{L}$, for instance, are all non-negative. If we number them in ascending order as in Section 6.13.2, so that $\lambda_1 \leq \lambda_2 \ldots \leq \lambda_n$, with $\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_n$ being the corresponding eigenvectors, then

$$(\lambda_n \mathbf{I} - \mathbf{L})\mathbf{v}_i = (\lambda_n - \lambda_i)\mathbf{v}_i\,,$$

(11.7)

and hence $\mathbf{v}_i$ is an eigenvector of $\lambda_n\mathbf{I} - \mathbf{L}$ with eigenvalue $\lambda_n - \lambda_i$. These eigenvalues are still all non-negative, but their order is reversed from those of the original Laplacian, so that the former smallest has become the new largest. Now we can calculate the eigenvector corresponding to the smallest eigenvalue of the Laplacian by finding the leading eigenvector of $\lambda_n\mathbf{I} - \mathbf{L}$ using the technique described in Section 11.1. We can also find the eigenvalue λ_1 by taking the measured value of λ_n- λ_1, subtracting λ_n, and reversing the sign. (Performing these calculations does require that we know the value of λ_n, so the complete calculation would be a two-stage process consisting of first finding the largest eigenvalue of $\mathbf{L}$, then using that to find the smallest.[165])

In this particular case, it would not in fact be very useful to calculate the smallest eigenvalue or its associated eigenvector since, as we saw in Section 6.13.2, the smallest eigenvalue of the Laplacian is always zero and the eigenvector is (1, 1, 1, . . .). However, if we can find the second-largest eigenvalue of a matrix we can use the same subtraction method also to find the second-smallest. And the second-smallest eigenvalue of the Laplacian is, as we have said, definitely of interest.

We can find the second-largest eigenvalue (and the corresponding eigenvector) using the following trick. Let $\mathbf{v}_1$ be the normalized eigenvector corresponding to the largest eigenvalue of a matrix $\mathbf{A}$, as found, for instance, by the power method of Section 11.1. Then we choose any starting vector $\mathbf{x}$ as before and define

$$\mathbf{y} = \mathbf{x} - (\mathbf{v}_1^T \mathbf{x})\mathbf{v}_1.$$

(11.8)

This vector has the property that

$$\begin{aligned}\mathbf{v}_i^T \mathbf{y} &= \mathbf{v}_i^T \mathbf{x} - (\mathbf{v}_1^T \mathbf{x})(\mathbf{v}_i^T \mathbf{v}_1) = \mathbf{v}_i^T \mathbf{x} - \mathbf{v}_1^T \mathbf{x}\,\delta_{i1} \\ &= \begin{cases} 0 & \text{if } i = 1, \\ \mathbf{v}_i^T \mathbf{x} & \text{otherwise,} \end{cases}\end{aligned}$$

(11.9)

where $\mathbf{v}_i$ is again the ith eigenvector of $\mathbf{A}$ and δ_{ij} is the Kronecker delta. In other words it is equal to $\mathbf{x}$ along the direction of every eigenvector of $\mathbf{A}$ except the leading eigenvector, in whose direction it has no component at all. This means that the expansion of $\mathbf{y}$ in terms of the eigenvectors of $\mathbf{A}$, which is given by $\mathbf{y} = \sum_{i=1}^{n} c_i \mathbf{v}_i$ with $c_i = \mathbf{v}_i^T \mathbf{y}$, has no term in $\mathbf{v}_1$, since $c_1 = \mathbf{v}_1^T \mathbf{y} = 0$. Thus

$$\mathbf{y} = \sum_{i=2}^{n} c_i \mathbf{v}_i,$$

(11.10)

with the sum starting at $i = 2$.

Now we use this vector $\mathbf{y}$ as the starting vector for repeated multiplication by $\mathbf{A}$, as before. After multiplying $\mathbf{y}$ by $\mathbf{A}$ a total of t times, we have

$$\mathbf{y}(t) = \mathbf{A}^t \mathbf{y}(0) = \kappa_2^t \sum_{i=2}^{n} c_i \left[\frac{\kappa_i}{\kappa_2}\right]^t \mathbf{v}_i.$$

(11.11)

The ratio κ_i/κ_2 is less than 1 for all $i > 2$ (assuming only a single eigenvalue of value κ_2) and hence in the limit of large t all terms in the sum disappear except the first so that $\mathbf{y}(t)$ tends to a multiple of $\mathbf{v}_2$ as $t \to \infty$. Normalizing this vector, we then have our result for $\mathbf{v}_2$.

This method has the same caveats as the original power method for the leading eigenvector, as well as one additional one: it is in practice possible for the vector $\mathbf{y}$, Eq. (11.8), to have a very

small component in the direction of $\mathbf{v}_1$. This can happen as a result of numerical error in the subtraction, or because our value for $\mathbf{v}_1$ is not exactly correct. If $\mathbf{y}$ does have a component in the direction of $\mathbf{v}_1$, then although it may start out small it will get magnified relative to the others when we multiply repeatedly by $\mathbf{A}$ and eventually it may come to dominate $\mathbf{y}(t)$, Eq. (11.11), or at least to contribute a sufficiently large term as to make the calculation of $\mathbf{v}_2$ inaccurate. To prevent this happening, we periodically perform a subtraction similar to that of Eq. (11.8), removing any component in the direction of $\mathbf{v}_1$ from $\mathbf{y}(t)$, while leaving the components in all other directions untouched. (The subtraction process is sometimes referred to as *Gram-Schmidt orthogonalization*—a rather grand name for a simple procedure. The repeated application of the process to prevent the growth of unwanted terms is called *reorthogonalization*.)

We could in theory extend this method to find further eigenvectors and eigenvalues of our matrix, but in practice the approach does not work well beyond the first couple of eigenvectors because of cumulative numerical errors. Moreover it is also slow because for each additional eigenvector we calculate we must carry out the entire repeated multiplication process again. In practice, therefore, if we wish to calculate anything beyond the first eigenvector or two, other methods are used.

11.1.3 EFFICIENT ALGORITHMS FOR COMPUTING ALL EIGENVALUES AND EIGENVECTORS OF MATRICES

If we wish to calculate all or many of the eigenvalues or eigenvectors of a matrix $\mathbf{A}$ then specialized techniques are needed. The most widely used such techniques involve finding an orthogonal matrix $\mathbf{Q}$ such that the similarity transform $\mathbf{T} = \mathbf{Q}^T\mathbf{A}\mathbf{Q}$ gives either a tridiagonal matrix (if $\mathbf{A}$ is symmetric) or a Hessenberg matrix (if $\mathbf{A}$ is asymmetric). If we can find such a transformation and if $\mathbf{v}_i$ is an eigenvector of $\mathbf{A}$ with eigenvalue κ_i, then, bearing in mind that for an orthogonal matrix $\mathbf{Q}^{-1} = \mathbf{Q}^T$, we have

$$\kappa_i \mathbf{Q}^T \mathbf{v}_i = \mathbf{Q}^T \mathbf{A} \mathbf{v}_i = \mathbf{T}\mathbf{Q}^T \mathbf{v}_i.$$

(11.12)

In other words, the vector $\mathbf{w}_i = \mathbf{Q}^T\mathbf{v}_i$ is an eigenvector of $\mathbf{T}$ with eigenvalue κ_i. Thus if we can find the eigenvalues of $\mathbf{T}$ and the corresponding eigenvectors, we automatically have the eigenvalues of $\mathbf{A}$ as well, and the eigenvectors of $\mathbf{A}$ are simply $\mathbf{v}_i = \mathbf{Q}\mathbf{w}_i$. Luckily there exist efficient numerical methods for finding the eigenvalues and eigenvectors of tridiagonal and Hessenberg matrices, such as the *QL algorithm* [273]. The QL algorithm takes time $O(n)$ to reach an answer for an $n \times n$ tridiagonal matrix and $O(n^2)$ for a Hessenberg one.

The matrix $\mathbf{Q}$ can be found in various ways. For a general symmetric matrix the *Householder algorithm* [273] can find $\mathbf{Q}$ in time $O(n^3)$. More often, however, we are concerned with sparse matrices, in which case there are faster methods. For a symmetric matrix, the *Lanczos algorithm* [217] can find $\mathbf{Q}$ in time $O(mn)$, where m is the number of network edges in an adjacency matrix, or more generally the number of non-zero elements in the matrix. For sparse matrices with $m \propto n$ this gives a running time of $O(n^2)$, considerably better than the Householder method. A similar method, the *Arnoldi algorithm* [217], can find $\mathbf{Q}$ for an asymmetric matrix.

Thus, combining the Lanczos and QL algorithms, we expect to be able to find all eigenvalues and eigenvectors of a sparse symmetric matrix in time $O(mn)$, which is as good as the worst-case run time of our direct multiplication method for finding just the leading eigenvector. (To be fair, the direct multiplication is much simpler, so its overall run time will typically be better than that of the combined Lanczos/QL algorithm, although the scaling with system size is the same.)

While there is certainly much to be gained by learning about the details of these algorithms, one rarely implements them in practice. Their implementation is tricky (particularly in the asymmetric case), and has besides already been done in a careful and professional fashion by many software developers. In practice, therefore, if one wishes to solve eigensystem problems for large networks, one typically turns to commercial or freely available implementations in professionally written software packages. Examples of such packages include Matlab, LAPACK, and Mathematica. We will not go into more detail here about the operation of these algorithms.

11.2 DIVIDING NETWORKS INTO CLUSTERS

We now turn to the topics that will occupy us for much of the rest of the chapter, *graph partitioning* and *community detection*.[166] Both of these terms refer to the division of the vertices of a network into groups, clusters, or communities according to the pattern of edges in the network. Most commonly one divides the vertices so that the groups formed are tightly knit with many edges inside groups and only a few edges between groups.

Consider Fig. 11.1, for instance, which shows patterns of collaborations between scientists in a university department. Each vertex in this network represents a scientist and links between vertices indicate pairs of scientists who have coauthored one or more papers together. As we can see from the figure, this network contains a number of densely connected clusters of vertices, corresponding to groups of scientists who have worked closely together. Readers familiar with the organization of university departments will not be surprised to learn that in general these clusters correspond, at least approximately, to formal research groups within the department.

But suppose one did not know how university departments operate and wished to study them. By constructing a network like that in Fig. 11.1 and then observing its clustered structure, one would be able to deduce the existence of groups within the larger department and by further investigation could probably quickly work out how the department was organized. Thus the ability to discover groups or clusters in a network can be a useful tool for revealing structure and organization within networks at a scale larger than that of a single vertex. In this particular case the network is small enough and sparse enough that the groups are easily visible by eye. Many of the networks that have engaged our interest in this book, however, are much larger or denser networks for which visual inspection is not a useful tool. Finding clusters in such networks is a task for computers and the algorithms that run on them.

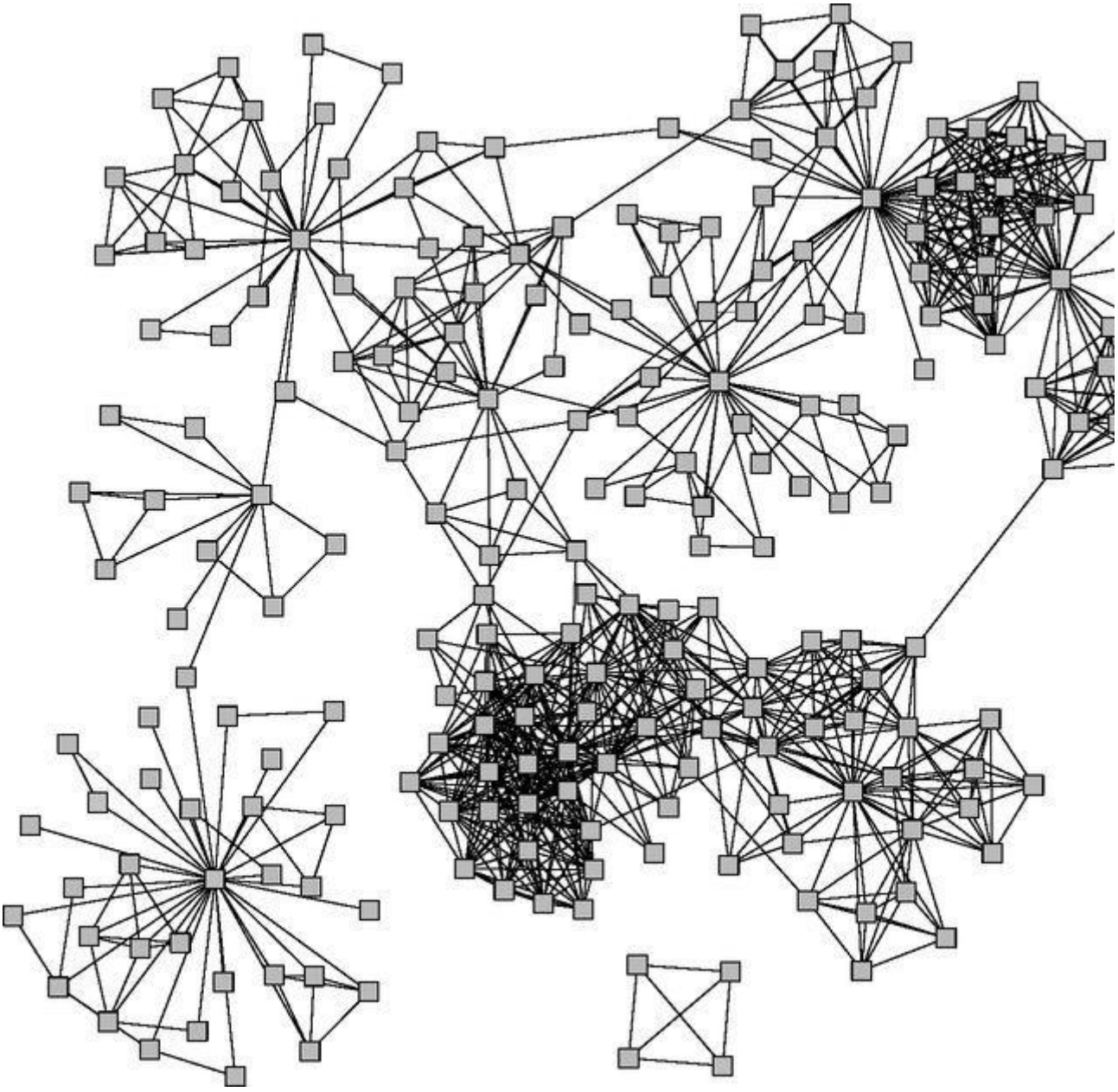


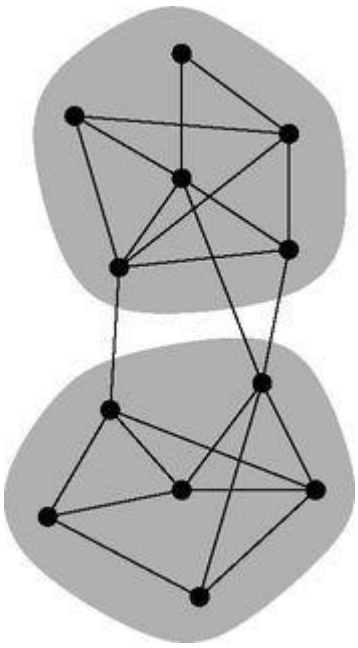
Figure 11.1: Network of coauthorships in a university department. The vertices in this network represent scientists in a university department, and edges links pairs of scientists who have coauthored scientific papers. The network has clear clusters or "community structure," presumably reflecting divisions of interests and research groups within the department.

11.2.1 PARTITIONING AND COMMUNITY DETECTION

There are a number of reasons why one might want to divide a network into groups or clusters, but they separate into two general classes that lead in turn to two corresponding types of computer algorithm. We will refer to these two types as *graph partitioning* and *community detection* algorithms. They are distinguished from one another by whether the number and size of the groups is fixed by the experimenter or whether it is unspecified.

Graph partitioning is a classic problem in computer science, studied since the 1960s. It is the problem of dividing the vertices of a network into a given number of non-overlapping groups of given sizes such that the number of edges between groups is minimized. The important point here is that the number and sizes of the groups are fixed. Sometimes the sizes are only fixed roughly—within a certain range, for instance—but they are fixed nonetheless. For instance, a simple and prototypical example of a graph partitioning problem is the problem of dividing a network into two groups of equal size, such that the number of edges between them is minimized.

Graph partitioning problems arise in a variety of circumstances, particularly in computer science, but also in pure and applied mathematics, physics, and of course in the study of networks themselves. A typical example is the numerical solution of network processes on a parallel computer.



Partition of a network into two groups of equal sizes.

In the last part of this book (Chapters 16 to 19) we will study processes that take place on networks, such as diffusion processes or the spread of diseases. These processes can be modeled mathematically by placing variables on the vertices of a network and evolving them according to equations that typically depend on the variables' current values and the values on neighboring vertices. The solution of such equations is often a laborious computational task, but it can be sped up by using a parallel computer, a computer with more than one processor or CPU. Many modern

personal computers have two or more processors and large research organizations sometimes use parallel computers with very many processors. Solutions of network equations can be spread across several processors by assigning to each processor the task of solving the equations on a subset of the vertices. For instance, on a two-processor desktop computer we might give a half of the vertices to each processor.

The catch is that, unless the network consists of totally unconnected components, some vertices on one processor are always going to have neighbors that are on the other processor and hence the solution of their equations involves variables whose value is known only to the other processor. To complete the solution, therefore, those values have to be transmitted from the one processor to the other at regular intervals throughout the calculation and this is typically a slow process (or at least it's slow compared to the dazzling speed of most other computer operations). The time spent sending messages between processors can, in fact, be the primary factor limiting the speed of calculations on parallel computers, so it is important to minimize interprocessor communication as much as possible. One way that we do this is by minimizing the number of pairs of neighboring vertices assigned to different processors.

Thus we want to divide up the vertices of the network into different groups, one for each processor, such that the number of edges between groups is minimized. Most often we want to assign an equal or roughly equal number of vertices to each processor so as to balance the workload among them. This is precisely a graph partitioning problem of the type described above.

The other type of cluster finding problem in networks is the problem we call community detection. Community detection problems differ from graph partitioning in that the number and size of the groups into which the network is divided are not specified by the experimenter. Instead they are determined by the network itself: the goal of community detection is to find the natural fault lines along which a network separates. The sizes of the groups are not merely unspecified but might in principle vary widely from one group to another. A given network might divide into a few large groups, many small ones, or a mixture of all different sizes.

The most common use for community detection is as a tool for the analysis and understanding of network data. We saw in Fig. 11.1 an example of a network for which a knowledge of the group structure might help us understand the organization of the underlying system. Figure 7.10 on page 221 shows another example of clusters of vertices, in a network of friendships between US high-school students. In this case the network splits into two clear groups, which, as described in Section 7.13, are primarily dictated by students' ethnicity, and this structure and others like it can give us clues about the nature of the social interactions within the community represented.

Community detection has uses in other types of networks as well. Clusters of nodes in a web graph for instance might indicate groups of related web pages. Clusters of nodes in a metabolic network might indicate functional units within the network.

Community detection is a less well-posed problem than graph partitioning. Loosely stated, it is the problem of finding the natural divisions of a network into groups of vertices such that there are many edges within groups and few edges between groups. What exactly we mean by "many" or "few," however, is debatable, and a wide variety of different definitions have been proposed, leading to a correspondingly wide variety of different algorithms for community detection. In this chapter we will focus mainly on the most widely used formulation of the problem, the formulation in terms of modularity optimization, but we will mention briefly a number of other approaches at the end of the chapter.

In summary, the fundamental difference between graph partitioning and community detection is that the number and size of the groups into which a network is divided is specified in graph partitioning but unspecified in community detection. However, there is also a difference between the goals of the two types of calculations. Graph partitioning is typically performed as a way of dividing up a network into smaller more manageable pieces, for example to perform numerical calculations. Community detection is more often used as a tool for understanding the structure of a network, for shedding light on large-scale patterns of connection that may not be easily visible in the raw network topology.

Notice also that in graph partitioning calculations the goal is usually to find the best division of a network, subject to certain conditions, regardless of whether any good division exists. If the performance of a calculation on a parallel computer, for example, requires us to divide a network

into pieces, then we had better divide it up. If there are no good divisions, then we must make do with the least bad one. With community detection, on the other hand, where the goal is normally to understand the structure of the network, there is no need to divide the network if no good division exists. Indeed if a network has no good divisions then that in itself may be a useful piece of information, and it would be perfectly reasonable for a community detection algorithm only to divide up networks when good divisions exist and to leave them undivided the rest of the time.

11.3 GRAPH PARTITIONING

In the next few sections we consider the graph partitioning problem and look at two well-known methods for graph partitioning. The first, the Kernighan–Lin algorithm, is not based on matrix methods (and therefore doesn't strictly belong in this chapter) but it provides a simple introduction to the partitioning problem and is worth spending a little time on. In Section 11.5 we look at a more sophisticated partitioning method based on the spectral properties of the graph Laplacian. This spectral partitioning method both is important in its own right and will also provide a basis for our discussion of community detection later in the chapter.

First, however, we address an important preliminary question: why does one need fancy partitioning algorithms at all? Partitioning is an easy problem to state, so is it not just as easy to solve?

11.3.1 WHY PARTITIONING IS HARD

The simplest graph partitioning problem is the division of a network into just two parts. Division into two parts is sometimes called *graph bisection*. Most of the algorithms we consider in this chapter are in fact algorithms for bisecting networks rather than for dividing them into arbitrary numbers of parts. This may at first appear to be a drawback, but in practice it is not, since if we can divide a network into two parts, then we can divide it into more than two by further dividing one or both of those parts. This repeated bisection is the commonest approach to the partitioning of networks into arbitrary numbers of parts.

Formally the graph bisection problem is the problem of dividing the vertices of a network into two non-overlapping groups of given sizes such that the number of edges running between vertices in different groups is minimized. The number of edges between groups is called the *cut size*.[167]

Simple though it is to describe, this problem is not easy to solve. One might imagine that one could bisect a network simply by looking through all possible divisions of the network into two parts of the required sizes and choosing the one with the smallest cut size. For all but the smallest of networks, however, this so-called *exhaustive search* turns out to be prohibitively costly in terms of computer time.

The number of ways of dividing a network of n vertices into two groups of n_1 and n_2 vertices respectively is $n!/(n_1!\,n_2!)$. Approximating the factorials using Stirling's formula $n! \simeq \sqrt{2\pi n}(n/\mathrm{e})^n$ and making use of the fact that $n_1 + n_2 = n$, we get

$$\frac{n!}{n_1!\,n_2!} \simeq \frac{\sqrt{2\pi n}(n/\mathrm{e})^n}{\sqrt{2\pi n_1}(n_1/\mathrm{e})^{n_1}\sqrt{2\pi n_2}(n_2/\mathrm{e})^{n_2}} = \frac{n^{n+1/2}}{n_1^{n_1+1/2}n_2^{n_2+1/2}}.$$

(11.13)

Thus, for instance, if we want to divide a network into two parts of equal size $\frac{1}{2}n$ the number of different ways to do it is roughly

$$\frac{n^{n+1/2}}{(n/2)^{n+1}} = \frac{2^{n+1}}{\sqrt{n}}.$$

(11.14)

So the amount of time required to look through all of these divisions will go up roughly exponentially with the size of the network. Unfortunately, the exponential is a very rapidly growing function of its argument, which means the partitioning task quickly leaves the realm of the possible at quite moderate values of n. Values up to about $n = 30$ are feasible with current computers, but go much beyond that and the calculation becomes intractable.

One might wonder whether it is possible to find a way around this problem. After all, brute-

force enumeration of all possible divisions of a network is not a very imaginative way to solve the partitioning problem. Perhaps one could find a way to limit one's search to only those divisions of the network that have a chance of being the best one? Unfortunately, there are some fundamental results in computer science that tell us that no such algorithm will ever be able to find the best division of the network in all cases. Either an algorithm can be clever and run quickly, but will fail to find the optimal answer in some (and perhaps most) cases, or it always finds the optimal answer but takes an impractical length of time to do it. These are the only options.[168]

This is not to say, however, that clever algorithms for partitioning networks do not exist or that they don't give useful answers. Even algorithms that fail to find the very best division of a network may still find a pretty good one, and for many practical purposes pretty good is good enough. The goal of essentially all practical partitioning algorithms is just to find a "pretty good" division in this sense. Algorithms that find approximate, but acceptable, solutions to problems in this way are called *heuristic algorithms* or just *heuristics*. All the algorithms for graph partitioning discussed in this chapter are heuristic algorithms.

11.4 THE KERNIGHAN-LIN ALGORITHM

The *Kernighan-Lin algorithm*, proposed by Brian Kernighan[169] and Shen Lin in 1970 [171], is one of the simplest and best known heuristic algorithms for the graph bisection problem. The algorithm is illustrated in Fig. 11.2.

We start by dividing the vertices of our network into two groups of the required sizes in any way we like. For instance, we could divide the vertices randomly. Then, for each pair (i, j) of vertices such that i lies in one of the groups and j in the other, we calculate how much the cut size between the groups would change if we were to interchange i and j, so that each was placed in the other group. Among all pairs (i, j) we find the pair that reduces the cut size by the largest amount or, if no pair reduces it, we find the pair that increases it by the smallest amount. Then we swap that pair of vertices. Clearly this process preserves the sizes of the two groups of vertices, since one vertex leaves each group and another joins. Thus the algorithm respects the requirement that the groups take specified sizes.

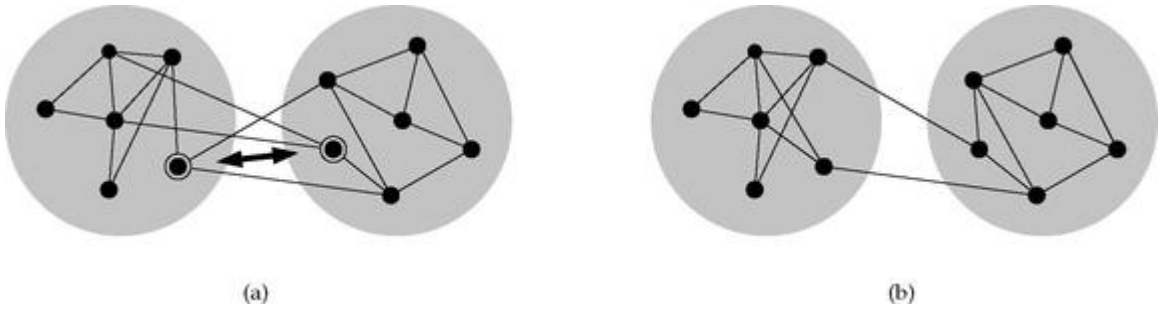


Figure 11.2: The Kernighan-Lin algorithm. (a) The Kernighan-Lin algorithm starts with any division of the vertices of a network into two groups (shaded) and then searches for pairs of vertices, such as the pair highlighted here, whose interchange would reduce the cut size between the groups. (b) The same network after interchange of the two vertices.

The process is then repeated, but with the important restriction that each vertex in the network can only be moved once. Once a vertex has been swapped with another it is not swapped again (at least not in the current round of the algorithm—see below). Thus, on the second step of the algorithm we consider all pairs of vertices excluding the two vertices swapped on the first step.

And so the algorithm proceeds, swapping on each step that pair that most decreases, or least increases, the number of edges between our two groups, until eventually there are no pairs left to be swapped, at which point we stop. (If the sizes of the groups are unequal then there will be vertices in the larger group that never get swapped, equal in number to the difference between the sizes of the groups.)

When all swaps have been completed, we go back through every state that the network passed through during the swapping procedure and choose among them the state in which the cut size takes its smallest value.[170]

Finally, this entire process is performed repeatedly, starting each time with the best division of the network found on the last time around and continuing until no improvement in the cut size occurs. The division with the best cut size on the last round is the final division returned by the

algorithm.

Once we can divide a network into two pieces of given size then, as we have said, we can divide into more than two simply by repeating the process. For instance, if we want to divide a network into three pieces of equal size, we would first divide into two pieces, one twice the size of the other, and then further divide the larger one into two equally sized halves. (Note, however, that even if the algorithm were able to find the optimal division of the network in each of these two steps, there would be no guarantee that we would end up with the optimal division of the network into three equal parts. Nonetheless, we do typically find a reasonably good division, which, as we have said, is often good enough. This point is discussed further in Section 11.9.)

Note that if we choose the initial assignment of vertices to groups randomly, then the Kernighan-Lin algorithm may not give the same answer if it is run twice on the same network. Two different random starting states could (though needn't necessarily) result in different divisions of the network. For this reason, people sometimes run the algorithm more than once to see if the results vary. If they do vary then among the divisions of the network returned on the different runs it makes sense to take the one with the smallest cut size.

As an example of the use of the Kernighan-Lin algorithm, consider Fig. 11.3, which shows an application of the algorithm to a mesh, a two-dimensional network of the type often used in parallel finite-element computations. Suppose we want to divide this network into two parts of equal size. Looking at the complete network in Fig. 11.3a there is no obvious division—there is no easy cut or bottleneck where the network separates naturally—but we must do the best we can. Figure 11.3b shows the best division found by the Kernighan-Lin algorithm, which involves cutting 40 edges in the network. Though it might not be the best possible division of the network, this is certainly good enough for many practical purposes.

The primary disadvantage of the Kernighan-Lin algorithm is that it is quite slow. The number of swaps performed during one round of the algorithm is equal to the smaller of the sizes of the two groups, which lies between zero and $\frac{1}{2}n$ in a network of n vertices. Thus there are $O(n)$ swaps in the worst case. For each swap we have to examine all pairs of vertices in different groups, of which there are, in the worst case, $\frac{1}{2}n \times \frac{1}{2}n = \frac{1}{4}n^2 = O(n^2)$. And for each of these we need to determine the change in the cut size if the pair is swapped.

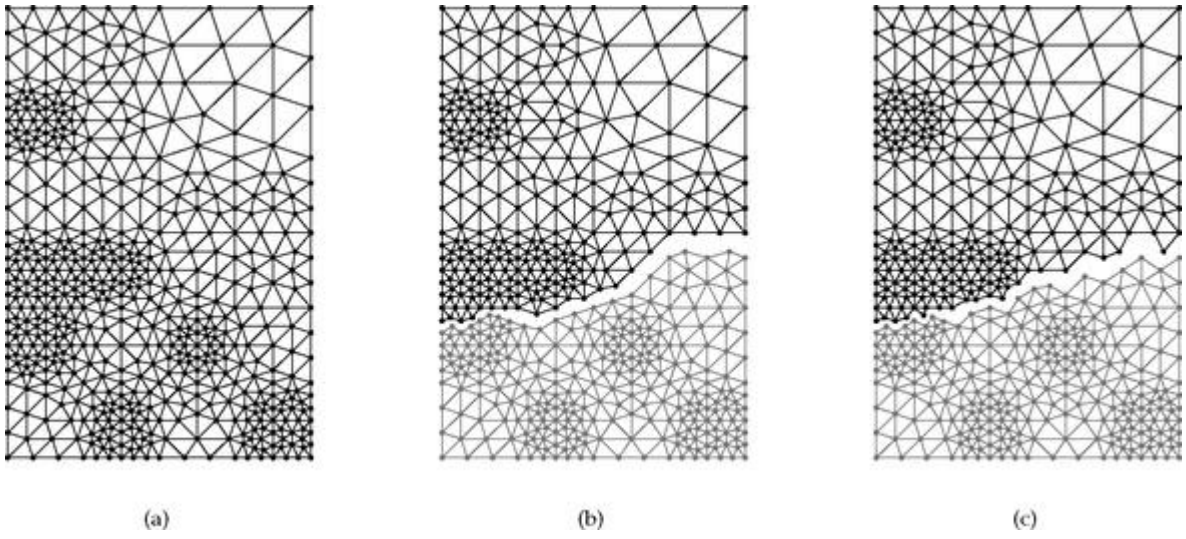


Figure 11.3: Graph partitioning applied to a small mesh network. (a) A mesh network of 547 vertices of the kind commonly used in finite element analysis. (b) The edges removed indicate the best division of the network into parts of 273 and 274 vertices found by the Kernighan-Lin algorithm. (c) The best division found by spectral partitioning. The network is from Bern *et al.* [35].

When a vertex i moves from one group to the other any edges connecting it to vertices in its current group become edges between groups after the swap. Let us suppose that there are k_i^{same} such edges. Similarly, any edges that i has to vertices in the other group, of which there are say k_i^{other}, become within-group edges after the swap, but with one exception. If i is being swapped with vertex j and there is an edge between i and j, then that edge lies between groups before the swap and still lies between groups after the swap. Thus the change in the cut size due to the movement of i is $k_i^{\text{other}} - k_i^{\text{same}} - A_{ij}$. A similar expression applies for vertex j also and the total change in cut size as a result of the swap is

$$\Delta = k_i^{\text{other}} - k_i^{\text{same}} + k_j^{\text{other}} - k_j^{\text{same}} - 2A_{ij}.$$

(11.15)

For a network stored in adjacency list form, the evaluation of this expression involves running through all the neighbors of i and j in turn, and hence takes time of order the average degree in the network, or $O(m/n)$, where m is, as usual, the total number of edges in the network.

Thus the total time for one round of the algorithm is $O(n \times n^2 \times m/n) = O(mn^2)$, which is $O(n^3)$ on a sparse network in which $m \propto n$ or $O(n^4)$ on a dense network. This in itself would already be quite bad, but we are not yet done. This time must be multiplied by the number of rounds the algorithm performs before the cut size stops decreasing. It is not well understood how the number of rounds required varies with network size. In typical applications the number is small, maybe five or ten for networks of up to a few thousand vertices, and larger networks are currently not possible because of the demands of the algorithm, so in practice the number of rounds is always small. Still, it seems quite unlikely that the number of rounds would actually increase as network size grows, and even if it remains constant the time complexity of the algorithm will still be $O(mn^2)$, which is relatively slow.

We can improve the running time of the algorithm a little by a couple of tricks. If we initially calculate and store the number of neighbors, k_i^{same} and k_i^{other}, that each vertex has within and between groups and update it every time a vertex is moved, then we save ourselves the time taken to recalculate these quantities on each step of the algorithm. And if we store our network in adjacency matrix form then we can tell whether two vertices are connected (and hence evaluate A_{ij}) in time $O(1)$. Together these two changes allow us to calculate Δ above in time $O(1)$ and improve the overall running time to $O(n^3)$. For a sparse graph this is the same as $O(mn^2)$, but for a dense one it gives us an extra factor of n.

Overall, however, the algorithm is quite slow. Even with $O(n^3)$ performance the algorithm is suitable only for networks up to a few hundreds or thousands of vertices, but not more.

11.5 SPECTRAL PARTITIONING

So are there faster methods for partitioning networks? There are indeed, although they are typically more complex than the simple Kernighan-Lin algorithm, and may be correspondingly more laborious to implement. In this section we discuss one of the most widely used methods, the *spectral partitioning* method of Fiedler [118, 271], which makes use of the matrix properties of the graph Laplacian. We describe the spectral partitioning method as applied to the graph bisection problem, the problem of dividing a graph into two parts of specified sizes. As discussed in the previous section, division into more than two groups is typically achieved by repeated bisection, dividing and subdividing the network to give groups of the desired number and size.

Consider a network of n vertices and m edges and a division of that network into two groups, which we will call group 1 and group 2. We can write the cut size for the division, i.e., the number of edges running between the two groups, as

$$R = \tfrac{1}{2} \sum_{\substack{i,j \text{ in} \\ \text{different} \\ \text{groups}}} A_{ij},$$

(11.16)

where the factor of $\frac{1}{2}$ compensates for our counting each edge twice in the sum.

Let us define a set of quantities s_i, one for each vertex i, which represent the division of the network thus:

$$s_i = \begin{cases} +1 & \text{if vertex } i \text{ belongs to group 1,} \\ -1 & \text{if vertex } i \text{ belongs to group 2.} \end{cases}$$

(11.17)

Then

$$\tfrac{1}{2}(1 - s_i s_j) = \begin{cases} 1 & \text{if } i \text{ and } j \text{ are in different groups,} \\ 0 & \text{if } i \text{ and } j \text{ are in the same group,} \end{cases}$$

(11.18)

which allows us to rewrite Eq. (11.16) as

$$R = \tfrac{1}{4}\sum_{ij} A_{ij}(1 - s_i s_j),$$

(11.19)

with the sum now over all values of i and j. The first term in the sum is

$$\sum_{ij} A_{ij} = \sum_i k_i = \sum_i k_i s_i^2 = \sum_{ij} k_i \delta_{ij} s_i s_j,$$

(11.20)

where k_i is the degree of vertex i as usual, δ_{ij} is the Kronecker delta, and we have made use of the fact that $\sum_j A_{ij} = k_i$ (see Eq. (6.19)) and $s_i^2 = 1$ (since $s_i = \pm 1$). Substituting back into Eq. (11.19) we then find that

$$R = \tfrac{1}{4}\sum_{ij}(k_i\delta_{ij} - A_{ij})s_i s_j = \tfrac{1}{4}\sum_{ij} L_{ij} s_i s_j,$$

(11.21)

where $L_{ij} = k_i\delta_{ij} - A_{ij}$ is the ijth element of the graph Laplacian matrix—see Eq. (6.44).

Equation (11.21) can be written in matrix form as

$$R = \tfrac{1}{4}\mathbf{s}^T\mathbf{L}\mathbf{s},$$

(11.22)

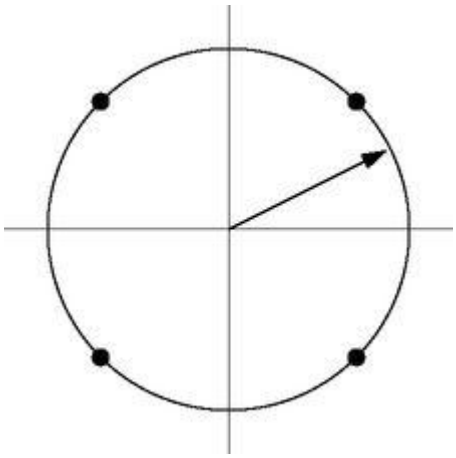
where **s** is the vector with elements s_i. This expression gives us a matrix formulation of the graph partitioning problem. The matrix **L** specifies the structure of our network, the vector **s** defines a division of that network into groups, and our goal is to find the vector **s** that minimizes the cut size (11.22) for given **L**.

You will probably not be surprised to learn that, in general, this minimization problem is not an easy one. If it were easy then we would have a corresponding easy way to solve the partitioning problem and, as discussed in Section 11.3.1, there are good reasons to believe that partitioning has no easy solutions.

What makes our matrix version of the problem hard in practice is that the s_i cannot take just any values. They are restricted to the special values ±1. If they were allowed to take any real values the problem would be much easier; we could just differentiate to the find the optimum.

This suggests a possible approximate approach to the minimization problem. Suppose we indeed

allow the s_i to take any values (subject to a couple of basic constraints discussed below) and then find the values that minimize R. These values will only be approximately the correct ones, since they probably won't be ±1, but they may nonetheless be good enough to give us a handle on the optimal partitioning. This idea leads us to the so-called *relaxation method*, which is one of the standard methods for the approximate solution of vector optimization problems such as this one. In the present context it works as follows.



The relaxation of the constraint allows **s** to point to any position on a hypersphere circumscribing the original hypercube, rather than just the corners of the hypercube.

The allowed values of the s_i are actually subject to two constraints. First, as we have said, each individual one is allowed to take only the values ±1. If we regard **s** as a vector in a Euclidean space then this constraint means that the vector always points to one of the 2^n corners of an n-dimensional hypercube centered on the origin, and always has the same length, which is $\sqrt{n}$. Let us relax the constraint on the vector's direction, so that it can point in any direction in its n-dimensional space. We will however still keep its length the same. (It would not make sense to allow the length to vary. If we did that then the minimization of R would have the obvious trivial solution $\mathbf{s} = 0$, which would tell us nothing.) So **s** will be allowed to take any value, but subject to the constraint that $|\mathbf{s}| = \sqrt{n}$, or equivalently

$$\sum_i s_i^2 = n.$$

(11.23)

Another way of putting this is that **s** can now point to any location on the surface of a hypersphere of radius $\sqrt{n}$ in our n-dimensional Euclidean space. The hypersphere includes the original allowed values at the corners of the hypercube, but also includes other points in between.

The second constraint on the s_i is that the numbers of them that are equal to +1 and −1 respectively must equal the desired sizes of the two groups. If those two sizes are n_1 and n_2, this second constraint can be written as

$$\sum_i s_i = n_1 - n_2.$$

(11.24)

or in vector notation

$$\mathbf{1}^T \mathbf{s} = n_1 - n_2,$$

(11.25)

where **1** is the vector (1, 1, 1, . . .) whose elements are all 1. We keep this second constraint unchanged in our relaxed calculations, so that our partitioning problem, in its relaxed form, is a problem of minimizing the cut size, Eq. (11.22), subject to the two constraints (11.23) and (11.24).

This problem is now just a standard piece of algebra. We differentiate with respect to the elements s_i, enforcing the constraints using two Lagrange multipliers, which we denote λ and 2μ (the extra 2 being merely for notational convenience):

$$\frac{\partial}{\partial s_i}\left[\sum_{jk} L_{jk} s_j s_k + \lambda\left(n - \sum_j s_j^2\right) + 2\mu\left((n_1 - n_2) - \sum_j s_j\right)\right] = 0.$$

(11.26)

Performing the derivatives, we then find that

$$\sum_j L_{ij} s_j = \lambda s_i + \mu,$$

(11.27)

or, in matrix notation

$$\mathbf{L}\mathbf{s} = \lambda \mathbf{s} + \mu \mathbf{1}.$$

(11.28)

We can calculate the value of μ by recalling that $\mathbf{1}$ is an eigenvector of the Laplacian with eigenvalue zero so that $\mathbf{L} \cdot \mathbf{1} = 0$ (see Section 6.13.2). Multiplying (11.28) on the left by $\mathbf{1}^T$ and making use of Eq. (11.25), we then find that $\lambda(n_1 - n_2) + \mu n = 0$, or

$$\mu = -\frac{n_1 - n_2}{n}\lambda.$$

(11.29)

If we define the new vector

$$\mathbf{x} = \mathbf{s} + \frac{\mu}{\lambda}\mathbf{1} = \mathbf{s} - \frac{n_1 - n_2}{n}\mathbf{1},$$

(11.30)

then Eq. (11.28) tells us that

$$\mathbf{L}\mathbf{x} = \mathbf{L}\left(\mathbf{s} + \frac{\mu}{\lambda}\mathbf{1}\right) = \mathbf{L}\mathbf{s} = \lambda\mathbf{s} + \mu\mathbf{1} = \lambda\mathbf{x},$$

(11.31)

where we have used $\mathbf{L} \cdot \mathbf{1} = 0$ again.

In other words, $\mathbf{x}$ is an eigenvector of the Laplacian with eigenvalue λ. We are still free to choose which eigenvector it is—any eigenvector will satisfy Eq. (11.31)—and clearly we should choose the one that gives the smallest value of the cut size R. Notice, however, that

$$\mathbf{1}^T\mathbf{x} = \mathbf{1}^T\mathbf{s} - \frac{\mu}{\lambda}\mathbf{1}^T\mathbf{1} = (n_1 - n_2) - \frac{n_1 - n_2}{n}n = 0,$$

(11.32)

where we have used Eq. (11.25). Thus $\mathbf{x}$ is orthogonal to $\mathbf{1}$, which means that, while it should be an eigenvector of $\mathbf{L}$, it cannot be the eigenvector (1, 1, 1, . . .) that has eigenvalue zero.

So which eigenvector should we choose? To answer this question we note that

$$R = \tfrac{1}{4}\mathbf{s}^T\mathbf{L}\mathbf{s} = \tfrac{1}{4}\mathbf{x}^T\mathbf{L}\mathbf{x} = \tfrac{1}{4}\lambda\mathbf{x}^T\mathbf{x}.$$

(11.33)

But from Eq. (11.30) we have

$$\begin{aligned}\mathbf{x}^T\mathbf{x} &= \mathbf{s}^T\mathbf{s} + \frac{\mu}{\lambda}(\mathbf{s}^T\mathbf{1} + \mathbf{1}^T\mathbf{s}) + \frac{\mu^2}{\lambda^2}\mathbf{1}^T\mathbf{1} \\ &= n - 2\frac{n_1 - n_2}{n}(n_1 - n_2) + \frac{(n_1 - n_2)^2}{n}n \\ &= 4\frac{n_1 n_2}{n},\end{aligned}$$

(11.34)

and hence

$$R = \frac{n_1 n_2}{n}\lambda.$$

(11.35)

Thus the cut size is proportional to the eigenvalue λ. Given that our goal is to minimize R, this means we should choose $\mathbf{x}$ to be the eigenvector corresponding to the smallest allowed eigenvalue of the Laplacian. All the eigenvalues of the Laplacian are non-negative (see Section 6.13.2). The smallest one is the zero eigenvalue that corresponds to the eigenvector (1, 1, 1, . . .) but we have already ruled this one out—$\mathbf{x}$ has to be orthogonal to this lowest eigenvector. Thus the best thing we can do is choose $\mathbf{x}$ proportional to the eigenvector $\mathbf{v}_2$ corresponding to the second lowest eigenvalue λ_2, with its normalization fixed by Eq. (11.34).

Finally, we recover the corresponding value of $\mathbf{s}$ from Eq. (11.30) thus:

$$\mathbf{s} = \mathbf{x} + \frac{n_1 - n_2}{n}\mathbf{1},$$

(11.36)

or equivalently

$$s_i = x_i + \frac{n_1 - n_2}{n}.$$

(11.37)

This gives us the optimal relaxed value of **s**.

As we have said, however, the real vector **s** is subject to the additional constraints that its elements take the values ±1 and moreover that exactly n_1 of them are +1 and the other n_2 are–1. Typically these constraints will prevent **s** from taking exactly the value given by Eq. (11.37). Let us, however, do the best we can and choose **s** to be as close as possible to our ideal value subject to its constraints, which we do by making the product

$$\mathbf{s}^T\Big(\mathbf{x} + \frac{n_1 - n_2}{n}\mathbf{1}\Big) = \sum_i s_i\Big(x_i + \frac{n_1 - n_2}{n}\Big)$$

(11.38)

as large as possible. The maximum of this expression is achieved by assigning $s_i = +1$ for the vertices with the largest (i.e., most positive) values of $x_i + (n_1 - n_2)/n$ and $s_i = -1$ for the remainder.

Note however that the most positive values of $x_i + (n_1 - n_2)/n$ are also the most positive values of x_i, which are in turn also the most positive elements of the eigenvector $\mathbf{v}_2$ (to which, as we have said, **x** is proportional). So after this moderately lengthy derivation we actually arrive at a very simple final prescription for dividing our network. We calculate the eigenvector $\mathbf{v}_2$, which has n elements, one for each vertex in the network, and place the n_1 vertices with the most positive elements in group 1 and the rest in group 2.

There is one further small subtlety. It is arbitrary which group we call group 1 and which we call group 2, and hence which one we assign to the more positive elements of the eigenvector and which to the more negative. Thus, if the sizes of the two groups are different there are two different ways of making the split—either the larger or the smaller group could correspond to the more positive values. (In the geometrical language of our vectors, this is equivalent to saying our eigenvector calculation might find the vector **x** that we actually want, or minus that vector—both are good eigenvectors of the Laplacian.) To get around this problem, we simply compute the cut size for both splits of the network and choose the one with the smaller value.

Thus our final algorithm is as follows:

1. Calculate the eigenvector $\mathbf{v}_2$ corresponding to the second smallest eigenvalue λ_2 of the graph Laplacian.
2. Sort the elements of the eigenvector in order from largest to smallest.
3. Put the vertices corresponding to the n_1 largest elements in group 1, the rest in group 2, and calculate the cut size.
4. Then put the vertices corresponding to the n_1 smallest elements in group 1, the rest in group 2, and recalculate the cut size.
5. Between these two divisions of the network, choose the one that gives the smaller cut size.

In Fig. 11.3c we show the result of the application of this method to the same mesh network that we studied in conjunction with the Kernighan-Lin algorithm. In this case the spectral method finds a division of the network very similar to that given by the Kernighan-Lin algorithm, although the cut size is slightly worse—the spectral method cuts 46 edges in this case, where the Kernighan-Lin algorithm cut only 40. This is typical of the spectral method. It tends to find divisions of a network that have the right general shape, but are not perhaps quite as good as those returned by other methods.

An advantage of the spectral approach, however, is its speed. The time-consuming part of the algorithm is the calculation of the eigenvector $\mathbf{v}_2$, which takes time $O(mn)$ using either the orthogonalization method or the Lanczos method (see Section 11.1.2), or $O(n^2)$ on a sparse network having $m \propto n$. This is one factor of n better than the $O(n^3)$ of the Kernighan-Lin algorithm, which makes the algorithm feasible for much larger networks. Spectral partitioning can be extended to networks of hundreds of thousands of vertices, where the Kernighan-Lin algorithm is restricted to networks of a few thousand vertices at most.

The second eigenvalue of the Laplacian has come up previously in this book in Section 6.13.3, where we saw that it is non-zero if and only if a network is connected. The second eigenvalue is for this reason sometimes called the *algebraic connectivity* of a network. In this section we have seen it again in another context, that of partitioning. What happens if a network is not connected and the second eigenvalue is zero? In that case, the two lowest eigenvalues are the same, and the corresponding eigenvectors are indeterminate—any mixture of two eigenvectors with the same eigenvalue is also an eigenvector. This is not however a serious problem. If the network is not connected, having more than one component, then usually we are interested either in partitioning one particular component, such as the largest component, or in partitioning all components individually, and so we just treat the components separately as connected networks according to the algorithm above.

The algebraic connectivity itself appears in our expression for the cut size, Eq. (11.35), and indeed is a direct measure of the cut size, being directly proportional to it, at least within the "relaxed" approximation used to derive the equation. Thus the algebraic connectivity is a measure of *how easily* a network can be divided. It is small for networks that have good cuts and large for those that do not. This in a sense is a generalization of our earlier result that the algebraic connectivity is non-zero for connected networks and zero for unconnected ones—we now see that *how* non-zero it is is a measure of how connected the network is.

11.6 COMMUNITY DETECTION

In the last few sections we looked at the problem of graph partitioning, the division of network vertices into groups of given number and size, so as to minimize the number of edges running between groups. A complementary problem, introduced in Section 11.2.1, is that of community detection, the search for the naturally occurring groups in a network regardless of their number or size, which is used primarily as a tool for discovering and understanding the large-scale structure of networks.

The basic goal of community detection is similar to that of graph partitioning: we want to separate the network into groups of vertices that have few connections between them. The important difference is that the number or size of the groups is not fixed. Let us focus to begin with on a very simple example of a community detection problem, probably *the* simplest, which is analogous to the graph bisection problems we examined in previous sections. We will consider the problem of dividing a network into just two non-overlapping groups or communities, as previously, but now without any constraint on the sizes of the groups, other than that the sum of the sizes should equal the size n of the whole network. Thus, in this simple version of the problem, the number of groups is still specified but their sizes are not, and we wish to find the "natural" division of the network into two groups, the fault line (if any) along which the network inherently divides, although we haven't yet said precisely what we mean by that, so that the question we're asking is not yet well defined.

Our first guess at how to tackle this problem might be simply to find the division with minimum cut size, as in the corresponding graph partitioning problem, but without any constraint on the sizes of our groups. However, a moment's reflection reveals that this will not work. If we divide a network into two groups with any number of vertices allowed in the groups then the optimum division is simply to put all the vertices in one of the groups and none of them in the other. This trivial division insures that the cut size between the two groups will be zero—there will be no edges between groups because one of the groups contains no vertices! As an answer to our community detection problem, however, it is clearly not useful.

One way to do better would be to impose loose constraints of some kind on the sizes of the groups. That is, we could allow the sizes of the groups to vary, but not too much. An example of this type of approach is *ratio cut partitioning* in which, instead of minimizing the standard cut size R, we instead minimize the ratio $R/(n_1 n_2)$, where n_1 and n_2 are the sizes of the two groups. The denominator $n_1 n_2$ has its largest value, and hence reduces the ratio by the largest amount, when n_1 and n_2 are equal $n_1 = n_2 = \frac{1}{2}n$. . For unequal group sizes the denominator becomes smaller the greater the inequality, and diverges when either group size becomes zero. This effectively eliminates solutions in which all vertices are placed in the same group, since such solutions never give the minimum value of the ratio, and biases the division towards those solutions in which the groups are of roughly equal size.

As a tool for discovering the natural divisions in a network, however, the ratio cut is not ideal. In particular, although it allows group sizes to vary it is still biased towards a particular choice, that of equally sized groups. More importantly, there is no principled rationale behind its definition. It works reasonably well in some circumstances, but there's no fundamental reason to believe it will give sensible answers or that some other approach will not give better ones.

An alternative strategy is to focus on a different measure of the quality of a division other than the simple cut size or its variants. It has been argued that the cut size is not itself a good measure because a good division of a network into communities is not merely one in which there are few edges between communities. On the contrary, the argument goes, a good division is one where

there are *fewer than expected* such edges. If we find a division of a network that has few edges between its groups, but nonetheless the number of such edges is about what we would have expected were edges simply placed at random in the network, then most people would say we haven't found anything significant. It is not the total cut size that matters, but how that cut size compares with what we expect to see.

In fact, in the conventional development of this idea one considers not the number of edges between groups but the number within groups. The two approaches are equivalent, however, since every edge that lies within a group necessarily does not lie between groups, so one can calculate one number from the other given the total number of edges in the network as whole. We will follow convention here and base our calculations on the numbers of within-group edges.

Our goal therefore will be to find a measure that quantifies how many edges lie within groups in our network relative to the number of such edges expected on the basis of chance. This, however, is an idea we have encountered before. In Section 7.13.1 we considered the phenomenon of assortative mixing in networks, in which vertices with similar characteristics tend to be connected by edges. There we introduced the measure of assortative mixing known as modularity, which has a high value when many more edges in a network fall between vertices of the same type than one would expect by chance. This is precisely the type of measure we need to solve our current community detection problem. If we consider the vertices in our two groups to be vertices of two types then good divisions of the network into communities are precisely those that have high values of the corresponding modularity.

Thus one way to detect communities in networks is to look for the divisions that have the highest modularity scores and in fact this is the most commonly used method for community detection. Like graph partitioning, modularity maximization is a hard problem (see Section 11.3.1). It is believed that, as with partitioning, the only algorithms capable of always finding the division with maximum modularity take exponentially long to run and hence are useless for all but the smallest of networks [54]. Instead, therefore, we turn again to heuristic algorithms, algorithms that attempt to maximize the modularity in an intelligent way that gives reasonably good results most of the time.

11.7 SIMPLE MODULARITY MAXIMIZATION

One straightforward algorithm for maximizing modularity is the analog of the Kernighan-Lin algorithm [245]. This algorithm divides networks into two communities starting from some initial division, such as a random division into equally sized groups. The algorithm then considers each vertex in the network in turn and calculates how much the modularity would change if that vertex were moved to the other group. It then chooses among the vertices the one whose movement would most increase, or least decrease, the modularity and moves it. Then it repeats the process, but with the important constraint that a vertex once moved cannot be moved again, at least on this round of the algorithm.

And so the algorithm proceeds, repeatedly moving the vertices that most increase or least decrease the modularity. Notice that in this algorithm we are not swapping pairs as we did in the Kernighan-Lin algorithm. In that algorithm we were required to keep the sizes of the groups constant, so for every vertex removed from a group we also had to add one. Now we no longer have such a constraint and so we can move single vertices on each step.

When all vertices have been moved exactly once, we go back over the states through which the network has passed and select the one with the highest modularity. We then use that state as the starting condition for another round of the same algorithm, and we keep repeating the whole process until the modularity no longer improves.

Figure 11.4 shows an example application of this algorithm to the "karate club" network of Zachary, which we encountered previously in Chapter 1 (see Fig. 1.2 on page 6). This network represents the pattern of friendships between members of a karate club at a North American university, as determined by direct observation of the club's members by the experimenter over a period of about two years. The network is interesting because during the period of observation a dispute arose among the members of the club over whether to raise the club's fees and as a result the club eventually split into two parts, of 18 and 16 members respectively, the latter departing to form their own club. The colors of the vertices in Fig. 11.4 denote the members of the two factions, while the shaded regions show the communities identified in the network by our vertex-moving algorithm. As we can see from the figure, the communities identified correspond almost perfectly to the known groups in the network. Just one vertex on the border between the groups is incorrectly assigned. Thus in this case our algorithm appears to have picked out structure of genuine sociological interest from an analysis of network data alone. It is precisely for results of this kind, that shed light on potentially important structural features of networks, that community detection methods are of interest.

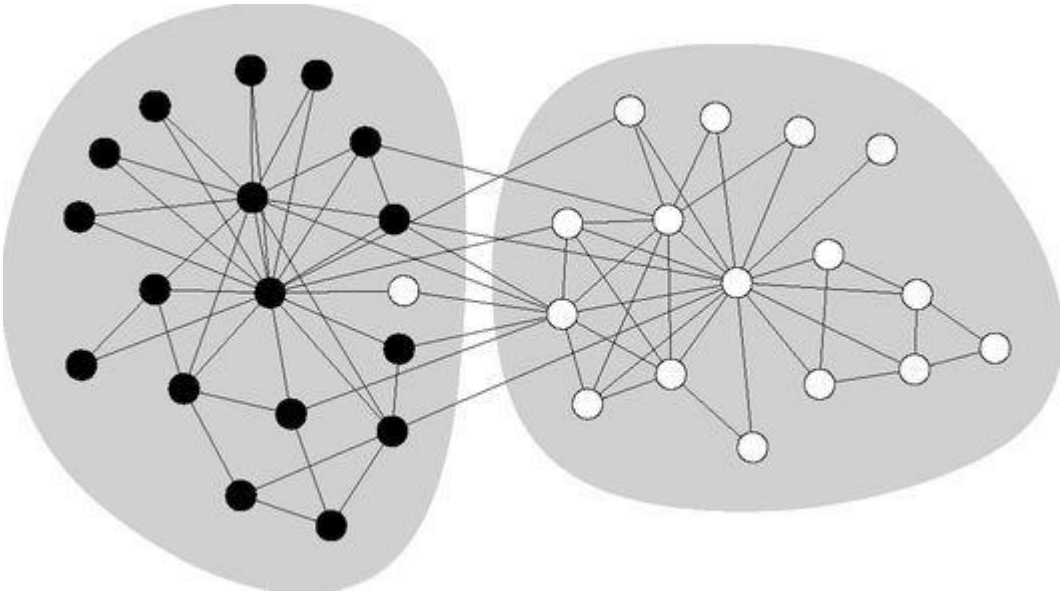


Figure 11.4: Modularity maximization applied to the karate club network. When we apply our vertex-moving modularity maximization algorithm to the karate club network, the best division found is the one indicated here by the two shaded regions, which split the network into two groups of 17 vertices each. This division is very nearly the same as the actual split of the network in real life (open and solid circles), following the dispute among the club's members. Just one vertex is classified incorrectly.

The vertex moving algorithm is also quite efficient. At each step of the algorithm we have to evaluate the modularity change due to the movement of each of $O(n)$ vertices, and each such evaluation, like the corresponding ones for the Kernighan-Lin algorithm, can be achieved in time $O(m/n)$ if the network is stored as an adjacency list. Thus each step takes time $O(m)$ and there are n steps in one complete round of the algorithm for a total time of $O(mn)$. This is considerably better than the $O(mn^2)$ of the Kernighan-Lin algorithm, and the algorithm is in fact one of the better of the many proposed algorithms for modularity maximization.[171] The fundamental reason for the algorithm's speed is that when moving single vertices we only have to consider $O(n)$ possible moves at each step, by contrast with the $O(n^2)$ possible swaps of vertex pairs that must be consider in a step of the Kernighan-Lin algorithm.

11.8 SPECTRAL MODULARITY MAXIMIZATION

Having seen in the previous section an algorithm for modularity maximization analogous to the Kernighan-Lin algorithm, it is natural to ask whether there also exists an analog for community detection of the spectral graph partitioning algorithm of Section 11.5. The answer is yes, there is indeed such an algorithm, as we now describe.

In Section 7.13.1 we wrote an expression for the modularity of a division of a network as follows (Eq. (7.69)):

$$Q = \frac{1}{2m}\sum_{ij}\left(A_{ij} - \frac{k_i k_j}{2m}\right)\delta(c_i, c_j) = \frac{1}{2m}\sum_{ij} B_{ij}\,\delta(c_i, c_j),$$

(11.39)

where c_i is the group or community to which vertex i belongs, $\delta(m, n)$ is the Kronecker delta, and

$$B_{ij} = A_{ij} - \frac{k_i k_j}{2m}.$$

(11.40)

Note that B_{ij} has the property

$$\sum_j B_{ij} = \sum_j A_{ij} - \frac{k_i}{2m}\sum_j k_j = k_i - \frac{k_i}{2m}\,2m = 0,$$

(11.41)

and similarly for sums over i. (We have made use of Eq. (6.20) in the second equality.) This property will be important shortly.

Let us again consider the division of a network into just two parts (we will consider the more general case later) and again represent such a division by the quantities

$$s_i = \begin{cases} +1 & \text{if vertex } i \text{ belongs to group 1,} \\ -1 & \text{if vertex } i \text{ belongs to group 2.} \end{cases}$$

(11.42)

We note that the quantity $\frac{1}{2}(s_i s_j + 1)$ is 1 if i and j are in the same group and zero otherwise, so that

$$\delta(c_i, c_j) = \tfrac{1}{2}(s_i s_j + 1).$$

(11.43)

Substituting this expression into Eq. (11.39), we find

$$Q = \frac{1}{4m} \sum_{ij} B_{ij} \, (s_i s_j + 1) = \frac{1}{4m} \sum_{ij} B_{ij} \, s_i s_j,$$

(11.44)

where we have used Eq. (11.41). In matrix terms we can write this as

$$Q = \frac{1}{4m} \mathbf{s}^T \mathbf{B} \mathbf{s},$$

(11.45)

where **s** is, as before, the vector with elements s_i, and **B** is the $n \times n$ matrix with elements B_{ij}, also called the *modularity matrix*.

Equation (11.45) is similar in form to our expression, Eq. (11.22), for the cut size of a network in terms of the graph Laplacian. By exploiting this similarity we can derive a spectral algorithm for community detection that is closely analogous to the spectral partitioning method of Section 11.5.

We wish to find the division of a given network that maximizes the modularity Q. That is, we wish to find the value of **s** that maximizes Eq. (11.45) for a given modularity matrix **B**. The elements of **s** are constrained to take values ±1, so that the vector always points to one of the corners of an n-dimensional hypercube, but otherwise there are no constraints on the problem. In particular, the number of elements with value +1 or −1 is not fixed as it was in the corresponding graph partitioning problem—the sizes of our communities are unconstrained.

As before, this optimization problem is a hard one, but it can be tackled approximately—and effectively—by a relaxation method. We relax the constraint that **s** must point to a corner of the hypercube and allow it to point in any direction, though keeping its length the same, meaning that it can take any real value subject only to the constraint that

$$\mathbf{s}^T\mathbf{s} = \sum_i s_i^2 = n.$$

(11.46)

The maximization is now a straightforward problem. We maximize Eq. (11.44) by differentiating, imposing the constraint with a single Lagrange multiplier β:

$$\frac{\partial}{\partial s_i}\left[\sum_{jk} B_{jk}s_j s_k + \beta\left(n - \sum_j s_j^2\right)\right] = 0.$$

(11.47)

When we perform the derivatives, this gives us

$$\sum_j B_{ij}s_j = \beta s_i,$$

(11.48)

or in matrix notation

$$\mathbf{B}\mathbf{s} = \beta\mathbf{s}.$$

(11.49)

In other words, **s** is one of the eigenvectors of the modularity matrix. Substituting (11.49) back into Eq. (11.45), we find that the modularity itself is given by

$$Q = \frac{1}{4m}\beta\mathbf{s}^T\mathbf{s} = \frac{n}{4m}\beta,$$

(11.50)

where we have used Eq. (11.46). For maximum modularity, therefore, we should choose **s** to be

the eigenvector $\mathbf{u}_1$ corresponding to the largest eigenvalue of the modularity matrix.

As before, we typically cannot in fact choose $\mathbf{s} = \mathbf{u}_1$, since the elements of $\mathbf{s}$ are subject to the constraint $s_i = \pm 1$. But we do the best we can and choose it as close to $\mathbf{u}_1$ as possible, which means maximizing the product

$$\mathbf{s}^T\mathbf{u}_1 = \sum_i s_i [\mathbf{u}_1]_i,$$

(11.51)

where $[\mathbf{u}_1]_i$ is the ith element of $\mathbf{u}_1$. The maximum is achieved when each term in the sum is non-negative, i.e., when

$$s_i = \begin{cases} +1 & \text{if } [\mathbf{u}_1]_i > 0, \\ -1 & \text{if } [\mathbf{u}_1]_i < 0. \end{cases}$$

(11.52)

In the unlikely event that a vector element is exactly zero, either value of s_i is equally good and we can choose whichever we prefer.

And so we are led the following very simple algorithm. We calculate the eigenvector of the modularity matrix corresponding to the largest (most positive) eigenvalue and then assign vertices to communities according to the signs of the vector elements, positive signs in one group and negative signs in the other.

In practice this method works very well. For example, when applied to the karate club network of Fig. 11.4 it works perfectly, classifying every one of the 34 vertices into the correct group.

One potential problem with the algorithm is that the matrix $\mathbf{B}$ is, unlike the Laplacian, not sparse, and indeed usually has all elements non-zero. At first sight, this appears to make the algorithm's complexity significantly worse than that of the normal spectral bisection algorithm; as discussed in Section 11.1.1, finding the leading eigenvector of a matrix takes time $O(mn)$, which is equivalent to $O(n^3)$ in a dense matrix, as opposed to $O(n^2)$ in a sparse one. In fact, however, by exploiting special properties of the modularity matrix it is still possible to find the eigenvector in time $O(n^2)$ on a sparse network. The details can be found in [246].

Overall, this means that the spectral method is about as fast as, but not significantly faster than, the vertex-moving algorithm of Section 11.7. Both have time complexity $O(n^2)$ on sparse networks.[172] There is, however, merit to having both algorithms. Given that all practical modularity maximizing algorithms are merely heuristics—clever perhaps, but not by any means guaranteed to perform well in all cases—having more than one fast algorithm in our toolkit is always a good thing.

11.9 DIVISION INTO MORE THAN TWO GROUPS

The community detection algorithms of the previous two sections both perform a limited form of community detection, the division of a network into exactly two communities, albeit of unspecified sizes. But "communities" are defined to be the natural groupings of vertices in networks and there is no reason to suppose that networks will in general have just two of them. They might have two, but they might have more than two, and we would like to be able to find them whatever their number. Moreover we don't, in general, want to have to specify the number of communities; that number should be fixed by the structure of the network and not by the experimenter.

In principle, the modularity maximization method can handle this problem perfectly well. Instead of maximizing modularity over divisions of a network into two groups, we should just maximize it over divisions into any number of groups. Modularity is supposed to be largest for the best division of the network, no matter how many groups that division possesses.

There are a number of community detection algorithms that take this "free maximization" approach to determining community number, and we discuss some of them in the following section. First, however, we discuss a simpler approach which is a natural extension of the methods of previous sections and of our graph partitioning algorithms, namely repeated bisection of a network. We start by dividing the network first into two parts and then we further subdivide those parts in to smaller ones, and so on.

One must be careful about how one does this, however. We cannot proceed as one can in the graph partitioning case and simply treat the communities found in the initial bisection of a network as smaller networks in their own right, applying our bisection algorithm to those smaller networks. The modularity of the complete network does not break up (as cut size does) into independent contributions from the separate communities and the individual maximization of the modularities of those communities treated as separate networks will not, in general, produce the maximum modularity for the network as a whole.

Instead, we must consider explicitly the change ΔQ in the modularity of the entire network upon further bisecting a community c of size n_c. That change is given by

$$\begin{aligned}\Delta Q &= \frac{1}{2m}\left[\frac{1}{2}\sum_{i,j\in c} B_{ij}(s_i s_j + 1) - \sum_{i,j\in c} B_{ij}\right] \\ &= \frac{1}{4m}\left[\sum_{i,j\in c} B_{ij} s_i s_j - \sum_{i,j\in c} B_{ij}\right] = \frac{1}{4m}\sum_{i,j\in c}\left[B_{ij} - \delta_{ij}\sum_{k\in c} B_{ik}\right] s_i s_j \\ &= \frac{1}{4m}\mathbf{s}^T\mathbf{B}^{(c)}\mathbf{s},\end{aligned} \tag{11.53}$$

where we have made use of $s_i^2 = 1$, and $\mathbf{B}^{(c)}$ is the $n_c \times n_c$ matrix with elements

$$B_{ij}^{(c)} = B_{ij} - \delta_{ij}\sum_{k\in c} B_{ik}.$$

(11.54)

Since Eq. (11.53) has the same general form as Eq. (11.45) we can now apply our spectral approach to this generalized modularity matrix, just as before, to maximize ΔQ, finding the leading eigenvector and dividing the network according to the signs of its elements.

In repeatedly subdividing a network in this way, an important question we need to address is at what point to halt the subdivision process. The answer is quite simple. Given that our goal is to maximize the modularity for the entire network, we should only go on subdividing groups so long as doing so results in an increase in the overall modularity. If we are unable to find any division of a community that results in a positive change ΔQ in the modularity, then we should simply leave that community undivided. The practical indicator of this situation is that our bisection algorithm will put all vertices in one of its two groups and none in the other, effectively refusing to subdivide the community rather than choose a division that actually decreases the modularity. When we have subdivided our network to the point where all communities are in this indivisible state, the algorithm is finished and we stop.

This repeated bisection method works well in many situations, but it is by no means perfect. A particular problem is that, as in the equivalent approach to graph partitioning, there is no guarantee that the best division of a network into, say, three parts, can be found by first finding the best division into two parts and then subdividing one of the two. Consider for instance the simple network shown in Fig. 11.5, which consists of eight vertices joined together in a line. The bisection of this network with highest modularity is the one shown in Fig. 11.5a, down the middle of the network, splitting it into two equally sized groups of four vertices each. The best modularity if the number of groups is unconstrained, however, is that shown in Fig. 11.5b, with three groups of sizes 3, 2, and 3, respectively. A repeated optimal bisection algorithm would never find the division in 11.5b because, having first made the bisection in 11.5a, there is no further bisection that will get us to 11.5b.

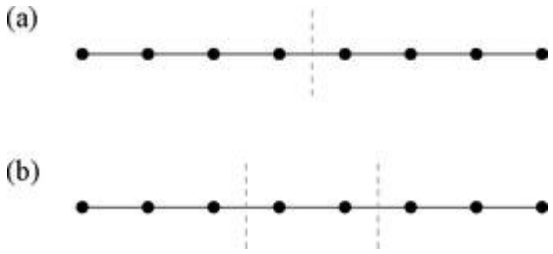


Figure 11.5: Division of a simple network by repeated maximization of the modularity. (a) The optimal bisection of this network of eight vertices and seven edges is straight down the middle. (b) The optimal division into an arbitrary number of groups is this division into three.

As mentioned above, an alternative method for dividing networks into more than two communities is to attempt to find directly the maximum modularity over divisions into any number of groups. This approach can, in principle, find better divisions than repeated bisection, but in practice is more complicated to implement and often runs slower. A number of promising methods have been developed, however, some of which are discussed in the next section.

11.10 OTHER MODULARITY MAXIMIZATION METHODS

There are a great variety of general algorithms for maximizing (or minimizing) functions over sets of states, and in theory any one of them could be brought to bear on the modularity maximization problem, thereby creating a new community detection algorithm. We describe briefly here three approaches that have met with some success. Each of these approaches attempts to maximize modularity over divisions into any number of communities of any sizes and thus to determine both the number and size of communities in the process.

One of the most widely used general optimization strategies is *simulated annealing*, which proceeds by analogy with the physics of slow cooling or "annealing" of solids. It is known that a hot system, such as a molten metal, will, if cooled sufficiently slowly to a low enough temperature, eventually find its *ground state*, that state of the system that has the lowest possible energy. Simulated annealing works by treating the quantity of interest—modularity in this case—as an energy and then simulating the cooling process until the system finds the state with the lowest energy. Since we are interested in finding the highest modularity, not the lowest, we equate energy in our case with *minus* the modularity, rather than with the modularity itself.

The details of the simulated annealing method are beyond the scope of this book, but the application to modularity maximization is a straightforward one and it appears to work very well [85, 150, 151, 215, 281]. For example, Danon *et al.* [85] performed an extensive test in which they compared the performance of a large number of different community detection algorithms on standardized tasks and found that the simulated annealing method gave the best results of any method tested. The main disadvantage of the approach is that it is slow, typically taking several times as long to reach an answer as competing methods do.

Another general optimization method is the genetic algorithm, a method inspired by the workings of biological evolution. Just as fitter biological species reproduce more and so pass on the genes that confer that fitness to future generations, so one can consider a population of different divisions of the same network and assign to each a "fitness" proportional to its modularity. Over a series of generations one simulates the preferential "reproduction" of highmodularity divisions, while those of low modularity die out. Small changes or mutations are introduced into the offspring divisions, allowing their modularity values either to improve or get worse and those that improve are more likely to survive in the next generation while those that get worse are more likely to be killed off. After many generations one has a population of divisions with good modularity and the best of these is the final division returned by the algorithm. Like simulated annealing the method appears to give results of high quality, but is slow, which restricts its use to networks of a few hundred vertices or fewer [295].

A third method makes use of a so-called *greedy algorithm*. In this very simple approach we start out with each vertex in our network in a one-vertex group of its own, and then successively amalgamate groups in pairs, choosing at each step the pair whose amalgamation gives the biggest increase in modularity, or the smallest decrease if no choice gives an increase. Eventually all vertices are amalgamated into a single large community and the algorithm ends. Then we go back over the states through which the network passed during the course of the algorithm and select the one with the highest value of the modularity. A naive implementation of this idea runs in time $O(n^2)$, but by making use of suitable data structures the run time can be improved to $O(n \log^2 n)$ on a sparse graph [71, 319]. Overall the algorithm works only moderately well: it gives reasonable divisions of networks, but the modularity values achieved are in general somewhat lower than those found by the other methods described here. On the other hand, the running time of the method may be the best of any current algorithm, and this is one of the few algorithms fast enough to work on the very largest networks now being explored. Wakita and Tsurumi [319] have given one example of an application to a network of more than five million vertices, something of a record for studies of this kind.

11.11 OTHER ALGORITHMS FOR COMMUNITY DETECTION

As we have seen, the problem of detecting communities in networks is a less well-posed one than the problem of graph partitioning. In graph partitioning the goal is clear: to find the division of a network with the smallest possible cut size. There is, by contrast, no universally agreed upon definition of what constitutes a good division of a network into communities. In the previous sections we have looked at algorithms based one particular definition in terms of the modularity function, but there are a number of other definitions in common use that lead to different algorithms. In the following sections we look briefly at a few of these other algorithms.